

# Maven 学习笔记

从第一次构建，到依赖仲裁、Fat JAR 与 Ratis ThirdParty

|        |        |        |        |        |
|--------|--------|--------|--------|--------|
| LAB 01 | LAB 02 | LAB 03 | LAB 04 | LAB 05 |
|--------|--------|--------|--------|--------|

为你的实际实验与提问定制

环境：Windows · JDK 11 · Maven Wrapper 3.9.9

整理日期：2026-07-16

# 使用说明

这不是 Maven 命令大全，而是一条围绕你已经完成的 Lab01-Lab05 建立起来的知识链。建议第一次顺序阅读，之后按“故障现象”和“速查卡”定点复习。

## 你的当前水平

你已经能通过实验观察构建目录、依赖树、JAR 内容和字节码，核心概念基本建立。当前需要重点巩固的是：Maven 模型与 JVM 类加载之间的边界，以及相似概念之间的准确区分。

# 目录

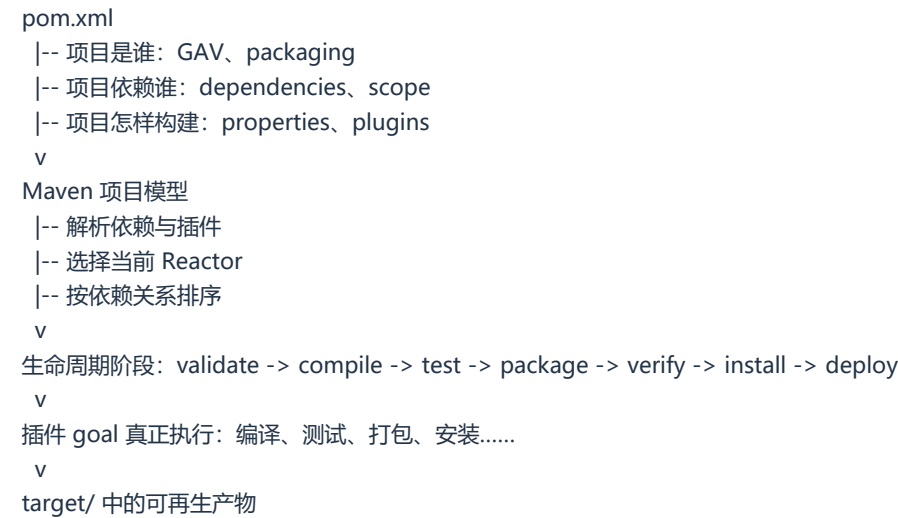
|                                               |           |
|-----------------------------------------------|-----------|
| <b>使用说明</b>                                   | <b>2</b>  |
| <b>目录</b>                                     | <b>3</b>  |
| <b>1. 一张图建立 Maven 总模型</b>                     | <b>6</b>  |
| 1.1 Windows 学习环境基线 . . . . .                  | 6         |
| <b>2. Lab01: 项目、坐标、目录与生命周期</b>                | <b>7</b>  |
| 2.1 GAV: 构件在 Maven 世界中的地址 . . . . .           | 7         |
| 2.2 Maven 为什么知道源码在哪里 . . . . .                | 7         |
| 2.3 compile、package、clean 的输入与输出 . . . . .    | 7         |
| 2.4 阶段与 goal 不要混为一谈 . . . . .                 | 8         |
| <b>3. Lab02: 依赖、scope、测试与仓库</b>               | <b>8</b>  |
| 3.1 dependency 与 plugin 的边界 . . . . .         | 8         |
| 3.2 常见 scope . . . . .                        | 8         |
| 3.3 本地仓库与三个关键阶段 . . . . .                     | 8         |
| 3.4 直接依赖与传递依赖 . . . . .                       | 8         |
| <b>4. Lab03: 多模块、Parent 与当前 Reactor</b>       | <b>9</b>  |
| 4.1 继承与聚合是两条关系 . . . . .                      | 9         |
| 4.2 什么是“当前 Reactor” . . . . .                 | 9         |
| 4.3 为什么 app JAR 里没有 GreetingService . . . . . | 9         |
| 4.4 pluginManagement 与 plugins . . . . .      | 9         |
| <b>5. Lab04: 依赖仲裁与二进制不兼容</b>                  | <b>10</b> |

|                                                         |           |
|---------------------------------------------------------|-----------|
| 5.1 实验中的依赖图 . . . . .                                   | 10        |
| 5.2 Maven 如何识别 “同一构件的不同版本” . . . . .                    | 10        |
| 5.3 仲裁规则：近者优先，同深度先声明 . . . . .                          | 10        |
| 5.4 为什么编译成功，运行却 NoSuchMethodError . . . . .             | 10        |
| 5.5 三类常见类加载错误 . . . . .                                 | 11        |
| <b>6. Maven 坐标冲突不等于重复 Class</b>                         | <b>11</b> |
| 6.1 Maven 默认解决的是坐标版本冲突 . . . . .                        | 11        |
| 6.2 JVM 如何处理同名 Class . . . . .                          | 11        |
| 6.3 Fat JAR 遇到重复路径 . . . . .                            | 11        |
| 6.4 如何发现 and 解决 . . . . .                               | 12        |
| <b>7. Lab05: Thin、Executable、Fat、Shade 与 Relocation</b> | <b>12</b> |
| 7.1 先把三个维度拆开 . . . . .                                  | 12        |
| 7.2 阶段一：Thin JAR . . . . .                              | 12        |
| 7.3 阶段二：Fat JAR . . . . .                               | 12        |
| 7.4 阶段三：Fat JAR 为什么仍会 NoSuchMethodError . . . . .       | 13        |
| 7.5 阶段四：Relocation 真正做了什么 . . . . .                     | 13        |
| 7.6 Shade 一词怎么记 . . . . .                               | 13        |
| <b>8. 映射回 Apache Ratis</b>                              | <b>14</b> |
| 8.1 为什么 Ratis 需要私有第三方命名空间 . . . . .                     | 14        |
| 8.2 为什么 ThirdParty 独立成构件 . . . . .                      | 14        |
| 8.3 skipShade 的准确理解 . . . . .                           | 14        |

|                                        |           |
|----------------------------------------|-----------|
| <b>9. 你的薄弱点与纠偏清单</b>                   | <b>15</b> |
| <b>10. Windows 排错操作手册</b>              | <b>15</b> |
| 10.1 先确认工具与当前项目 . . . . .              | 15        |
| 10.2 再确认依赖解析结果 . . . . .               | 15        |
| 10.3 然后检查真实 JAR . . . . .              | 15        |
| 10.4 按错误类型定位 . . . . .                 | 16        |
| <b>11. 复习卡：必须能脱口而出的 20 题</b>           | <b>16</b> |
| <b>12. 巩固练习与参考答案</b>                   | <b>18</b> |
| 练习 A：预测而不是先运行 . . . . .                | 18        |
| 练习 B：画出当前 Reactor . . . . .            | 18        |
| 练习 C：区分两类冲突 . . . . .                  | 18        |
| 练习 D：用证据定位 NoSuchMethodError . . . . . | 18        |
| 12.1 推荐复习节奏 . . . . .                  | 18        |
| <b>结语</b>                              | <b>19</b> |

# 1. 一张图建立 Maven 总模型

先不要把 Maven 想成“下载 JAR 的工具”。它的核心工作是读取项目模型，然后按生命周期调用插件。



最值得记住的一句话:

**pom.xml 描述“项目是谁、依赖谁、怎样构建”；Maven 按生命周期调用插件，把标准目录中的输入变成 target 中的产物。**

## 1.1 Windows 学习环境基线

| 项目    | 建议                                                        | 原因                       |
|-------|-----------------------------------------------------------|--------------------------|
| JDK   | IDE Project SDK、Maven Runner JRE、命令行 JAVA_HOME 均使用 JDK 11 | 减少 IDE 与命令行结果不一致         |
| Maven | 优先使用 .\mvnw.cmd                                           | 项目固定 Maven 3.9.9，换机器仍可复现 |
| 编码    | 项目与文件均使用 UTF-8                                            | 避免中文源码、日志与文档乱码           |
| 实验目录  | ratls-labs 与 ratls 源码仓库同级                                 | 实验不会污染真实项目工作树            |

```
cd D:\software\idea_workstation\ratls_20260610\ratls-labs
.\mvnw.cmd -version
java -version
```

为什么已有 mvn 仍推荐 mvnw?

mvn 使用机器上全局安装的 Maven；mvnw.cmd 按项目配置启动指定版本。Wrapper 不是能力更强，而是把工具版本也纳入项目的可复现约定。

## 2. Lab01：项目、坐标、目录与生命周期

### 2.1 GAV：构件在 Maven 世界中的地址

| 字段         | 含义      | 实验示例         |
|------------|---------|--------------|
| groupId    | 组织或命名空间 | study.ratis  |
| artifactId | 构件名称    | hello-maven  |
| version    | 构件版本    | 1.0-SNAPSHOT |

三项合起来是 study.ratis:hello-maven:1.0-SNAPSHOT。SNAPSHOT 表示开发中的可变版本。构建得到的文件名通常采用 artifactId-version.jar。

### 2.2 Maven 为什么知道源码在哪里

```
src/main/java    生产 Java 源码
src/main/resources 生产资源
src/test/java    测试 Java 源码
src/test/resources 测试资源
target/classes   编译后的生产 class
target/test-classes 编译后的测试 class
```

这叫“约定优于配置”。Java 文件里的 package 决定其在源码根目录下的相对路径。例如 package study.ratis.lab 对应 study/ratis/lab。

### 2.3 compile、package、clean 的输入与输出

| 命令/阶段   | 主要动作        | 重点输出                                 |
|---------|-------------|--------------------------------------|
| compile | 编译生产源码      | target/classes/**/*.class            |
| test    | 编译并运行测试     | target/test-classes、surefire-reports |
| package | 执行前置阶段并形成构件 | target/*.jar                         |
| clean   | 删除构建输出      | target 被删除                           |

执行 package 时不需要先手工执行 compile。生命周期阶段具有顺序关系，Maven 会从起点依次执行到你指定的阶段。

#### 你的一个关键疑问

为什么 -cp 有时接目录，有时接 JAR？因为 classpath 的每一个条目都是“类搜索根”。目录条目从目录根按包路径找 .class；JAR 条目在压缩包内部按同样的包路径找 .class。两者只是类的容器不同。

```
# 目录是类搜索根
```

```
java -cp .\target\classes study.ratis.lab.HelloMaven
```

```
# JAR 也可以是类搜索根
```

```
java -cp .\target\hello-maven-1.0-SNAPSHOT.jar study.ratis.lab.HelloMaven
```

## 2.4 阶段与 goal 不要混为一谈

compile、package、install 是生命周期阶段；compiler:compile、jar:jar、install:install 是插件 goal。阶段本身描述“走到哪里”，绑定到阶段的插件 goal 才真正执行工作。dependency:tree 则是直接调用 dependency 插件的 tree goal，它不属于默认生命周期阶段。

## 3. Lab02：依赖、scope、测试与仓库

### 3.1 dependency 与 plugin 的边界

| 概念         | 服务对象       | 是否进入项目 classpath | 例子                      |
|------------|------------|------------------|-------------------------|
| dependency | 你的 Java 程序 | 按 scope 决定       | JUnit、Guava             |
| plugin     | Maven 构建过程 | 通常不作为业务依赖        | compiler、surefire、shade |

简单记忆：dependency 给项目代码提供 class；plugin 给 Maven 提供构建动作。Java 代码 import JUnit，但不会 import maven-surefire-plugin。

### 3.2 常见 scope

| scope        | 主代码编译 | 测试 | 运行      | 是否通常传递 |
|--------------|-------|----|---------|--------|
| compile (默认) | 是     | 是  | 是       | 是      |
| test         | 否     | 是  | 否       | 否      |
| runtime      | 否     | 是  | 是       | 是      |
| provided     | 是     | 是  | 由运行环境提供 | 否      |

#### 修正一个容易说过头的表述

“compile scope 会参与打包”不等于“普通 JAR 会把依赖 class 打进去”。compile 依赖会参与依赖解析与运行 classpath；是否把依赖内容合并进当前 JAR，取决于是否使用 Shade/Assembly 等打包插件。

### 3.3 本地仓库与三个关键阶段

package -> 构件留在当前项目 target/  
install -> 在 package 之后，安装到 ~/.m2/repository/  
deploy -> 在 install 之后，发布到远程仓库

本地仓库不是普通的“JAR 收纳文件夹”，而是按坐标组织的构件缓存。路径大致为 groupId 路径 /artifactId/version。POM 也会一同保存，因为传递依赖信息来自 POM。

### 3.4 直接依赖与传递依赖

```
calculator
├─ junit-jupiter      直接依赖：你在 POM 中声明
├─ junit-jupiter-api  传递依赖：由直接依赖带入
└─ junit-jupiter-engine 传递依赖
```

dependency:tree 是理解真实 classpath 的第一工具。它展示的是依赖图经过 scope、exclusion、optional 与版本仲裁后的结果，而不是简单复述 POM。

## 4. Lab03：多模块、Parent 与当前 Reactor

### 4.1 继承与聚合是两条关系

| POM 元素           | 关系 | 解决的问题             |
|------------------|----|-------------------|
| 子 POM 的 parent   | 继承 | 配置、属性、版本从哪里来      |
| 聚合 POM 的 modules | 聚合 | 一次 Maven 命令包含哪些项目 |

一个 POM 经常同时是 parent 和 aggregator，但概念上必须分开。packaging=pom 表示它的主要产物是项目模型与构建组织，而不是业务 JAR。

### 4.2 什么是“当前 Reactor”

Reactor 是本次 Maven 命令在内存中选中的一组 MavenProject、它们之间的依赖图以及据此计算出的构建顺序。它不是目录，不是仓库，也不是永久保存的东西。命令结束，本次 Reactor 就结束。

执行父 POM: clean package

当前 Reactor = parent + greeting-library + greeting-app

执行父 POM: -pl greeting-app package

当前 Reactor = greeting-app

执行父 POM: -pl greeting-app -am package

当前 Reactor = greeting-library + greeting-app (以及必要的聚合项目)

#### 为什么加 -am 后 package 能成功?

-pl 只选择 app 时，library 不在当前 Reactor，Maven 只能去本地/远程仓库找它。加 -am 后，Maven 把 app 依赖的 Reactor 模块也加入本次构建；app 可直接使用本次 Reactor 中 library 的输出，因此即使本地仓库尚未 install 该坐标，也可以 package 成功。

### 4.3 为什么 app JAR 里没有 GreetingService

因为 -am 解决的是参与本次构建和编译 classpath 的问题，不是“把依赖复制进 app JAR”。普通 maven-jar-plugin 只打包 app 自己的 target/classes 和资源。greeting-library 仍然是独立构件。

-am: 让依赖模块参与当前 Reactor

jar: 只打包当前模块内容

shade: 才可能把已解析依赖的内容合并进一个 JAR

### 4.4 pluginManagement 与 plugins

| 位置               | 作用                        |
|------------------|---------------------------|
| pluginManagement | 统一插件版本和默认配置；通常不会凭此自动执行    |
| plugins          | 当前模块实际声明使用插件，使其 goal 参与构建 |

## 5. Lab04：依赖仲裁与二进制不兼容

### 5.1 实验中的依赖图

```
app-conflict
+- consumer-a -> demo-thirdparty:1.0 -> format()
\ - consumer-b -> demo-thirdparty:2.0 -> format() + upper()
```

consumer-a 编译时看到 v1；consumer-b 编译时看到 v2，所以两个 consumer 都能独立编译。到 app 解析完整依赖图时，Maven 必须从同一构件的多个版本中选一个。

### 5.2 Maven 如何识别“同一构件的不同版本”

简化理解时，可把冲突身份看作 groupId + artifactId + type/extension + classifier。当这些相同而 version 不同时，Maven 把它们视为同一个依赖身份的不同版本，进入版本仲裁。scope 不负责定义构件身份。

| 坐标 A              | 坐标 B        | Maven 是否版本仲裁    |
|-------------------|-------------|-----------------|
| g:a:jar:1.0       | g:a:jar:2.0 | 是：身份相同，版本不同     |
| g:a:jar:1.0       | g:b:jar:1.0 | 否：artifactId 不同 |
| g:a:jar:1.0       | x:a:jar:1.0 | 否：groupId 不同    |
| g:a:jar:tests:1.0 | g:a:jar:1.0 | 否：classifier 不同 |

### 5.3 仲裁规则：近者优先，同深度先声明

第一规则是离当前项目的依赖路径更近者优先。只有路径深度相同，声明顺序才作为常见的决胜因素。dependencyManagement 可以显式统一版本，使选择不再依赖偶然的传递路径。

```
demo-thirdparty:2.0 (omitted for conflict with 1.0)
```

含义：v2 出现在依赖图中，但仲裁落败，没有进入最终解析结果。  
它不表示两个版本同时进入 classpath。

### 5.4 为什么编译成功，运行却 NoSuchMethodError

1. consumer-b 编译：使用 v2，upper() 存在 -> 成功
2. app 编译：只调用 ConsumerB.message() -> 成功
3. app 解析依赖：v1 获胜，v2 被 omitted
4. JVM 运行：成功找到 v1 TextFormatter
5. JVM 解析 upper()：v1 没有该方法 -> NoSuchMethodError

app 编译器不会重新打开所有依赖库的字节码，替它们重新验证每一个方法符号引用。问题要到 JVM 实际解析 consumer-b 字节码中的 upper() 调用时才暴露。

## 5.5 三类常见类加载错误

| 错误                     | 核心含义                  | 本实验对应                  |
|------------------------|-----------------------|------------------------|
| ClassNotFoundException | 代码主动按类名加载，但类找不到       | 常见于反射/Class.forName    |
| NoClassDefFoundError   | JVM 运行已编译代码时，所需类定义不可用 | thin JAR 缺 DemoLibrary |
| NoSuchMethodError      | 类存在，但运行时版本没有字节码要求的方法  | Lab04 的 v1 缺 upper()   |

## 6. Maven 坐标冲突不等于重复 Class

这是你在完成 Lab05 后提出的关键问题，也是从 Maven 进入 JVM 世界的分界线。

### 6.1 Maven 默认解决的是坐标版本冲突

Maven 通常不会打开每个 JAR，逐项比较其中的 class。只要两个 Artifact 的冲突身份不同，它们就可以同时出现在依赖结果中。

```
com.company-a:utility-a:1.0
com.company-b:utility-b:1.0
```

两个坐标完全不同，但两个 JAR 都包含：  
com/example/util/StringUtils.class

#### 结论

这是 Java classpath 层面的重复类，不是 Maven 坐标层面的版本冲突。Maven 默认不会替你仲裁掉其中一个。

### 6.2 JVM 如何处理同名 Class

在传统 classpath 模式下，ClassLoader 通常按搜索顺序找到第一个满足全限定类名的定义并加载。类的身份至少与“类加载器 + 二进制类名”有关。两个 JAR 中路径完全相同的 class，不会因为来自不同 Maven 坐标就自动成为两个类。

```
java -cp "utility-a.jar;utility-b.jar;app.jar" com.example.App
# 通常先命中 utility-a.jar 中的 StringUtils
```

```
java -cp "utility-b.jar;utility-a.jar;app.jar" com.example.App
# 顺序改变后，可能先命中 utility-b.jar 中的 StringUtils
```

因此结果可能随 IDE、插件、容器或启动脚本生成的 classpath 顺序改变。这种“谁先被加载谁生效”的隐式行为不应作为解决方案。

### 6.3 Fat JAR 遇到重复路径

合并 JAR 时，同一个归档路径不能可靠地保存两份不同内容。Shade 可能警告 overlapping classes/resources，最终只留下其中一个，具体结果受处理顺序和插件配置影响。构建成功也不能证明选中的 class 正确。

## 6.4 如何发现和解决

| 动作                                         | 用途                             |
|--------------------------------------------|--------------------------------|
| <code>mvn dependency:tree -Dverbose</code> | 检查坐标、路径、scope 与 omitted        |
| <code>jar tf xxx.jar</code>                | 确认某个 JAR 实际包含哪些 class          |
| <code>javap -classpath ... -c 类名</code>    | 确认字节码调用的真实类/方法符号               |
| Maven Enforcer 的重复类规则                      | 让构建扫描依赖并对 duplicate classes 失败 |
| exclusion / 升级 / 统一版本                      | 移除不需要或不兼容的构件                   |
| Shade relocation                           | 改变一方全限定类名，实现命名空间隔离             |

## 7. Lab05: Thin、Executable、Fat、Shade 与 Relocation

### 7.1 先把三个维度拆开

| JAR 形态       | 有 Main-Class | 包含依赖 class | 典型运行方式                                      |
|--------------|--------------|------------|---------------------------------------------|
| 普通 thin JAR  | 不一定          | 否          | <code>java -cp ... 完整类名</code>              |
| 可执行 thin JAR | 是            | 否          | <code>java -jar</code> 可启动，但依赖仍须解决          |
| fat/uber JAR | 不一定          | 是          | 若有 Main-Class，可 <code>java -jar</code> 独立运行 |

“可执行”描述入口；“fat”描述内容。二者是两个不同维度。一个 JAR 可以有 Main-Class 但缺依赖，也可以包含依赖却没有入口。

### 7.2 阶段一：Thin JAR

```
thin-app.jar      只含 ThinApp.class
demo-library.jar  含 DemoLibrary.class
```

```
java -jar thin-app.jar      -> NoClassDefFoundError
java -cp "thin-app.jar;demo-library.jar" ... -> 成功
```

关键观察：package 成功只表示产物按当前打包规则生成成功，不表示运行时依赖已经全部进入单个 JAR。

### 7.3 阶段二：Fat JAR

maven-shade-plugin 在 package 阶段读取当前模块 class 和 Maven 已解析出的依赖 JAR，并合并内容。original-fat-app...jar 是 shade 前的普通 JAR 备份；主产物被替换为合并后的 fat JAR。

```
fat-app-1.0-SNAPSHOT.jar
study/.../FatApp.class
study/.../DemoLibrary.class
```

解决：部署便利、classpath 组装便利  
不保证：版本兼容、重复 Class 正确选择、资源无冲突

## 7.4 阶段三：Fat JAR 为什么仍会 NoSuchMethodError

顺序是先依赖解析与版本仲裁，再由 Shade 合并已选中的构件。v2 已经 omitted，Shade 不会凭空把它恢复。因此 fat-conflict-app 中只有 v1 TextFormatter，consumer-b 运行时仍找不到 upper()。

可以把关系记成：

**Fat JAR 是依赖仲裁结果的打包表现，不是依赖仲裁的替代品。**

## 7.5 阶段四：Relocation 真正做了什么

原名：study.ratis.lab.thirdparty.TextFormatter  
新名：study.ratis.lab.internal.shaded.thirdparty.TextFormatter

Shade 修改：

1. JAR 中 class 条目路径
2. class 文件内部的类名
3. 使用该类的字节码符号引用

ConsumerBShaded 源码仍 import 原包，是因为 javac 发生在 Shade 之前：先针对原始 v2 编译，再在 package 阶段改写产物。javap 看到 relocated 名称，才是最直接的字节码证据。

### 对你第 6 题答案的精确修正

app-safe 能同时存在两个 TextFormatter，不是因为 Maven 把它们“认为成两个不同依赖”，而是 relocation 让它们拥有不同的全限定类名。JVM 因而看到两个不同的类；被 relocation 的 v2 往往已经嵌入 consumer-b-shaded 构件。

## 7.6 Shade 一词怎么记

shade 直译可理解为“遮蔽、加阴影”。在 Maven 语境中，它指重新打包并可能改写依赖内容；relocation 是 Shade 能做的一项具体转换，不要把 Shade 与 Relocation 当成完全同义词。

## 8. 映射回 Apache Ratis

### 8.1 为什么 Ratis 需要私有第三方命名空间

宿主系统: io.netty.\* -> Netty B

Ratis: org.apache.ratis.thirdparty.io.netty.\* -> Ratis 隔离后的 Netty A

Ratis 会作为库嵌入 Hadoop 等大型系统。宿主与 Ratis 可能需要不同版本的 Netty、gRPC、Protobuf。如果都暴露原包名，就可能发生 Lab04 的二进制不兼容或重复 Class。Relocation 通过改变 Ratis 一侧的全限定类名，让两个版本在 JVM 中共存。

### 8.2 为什么 ThirdParty 独立成构件

- 把 relocation 后的依赖集中封装，减少对使用者 classpath 的污染与版本碰撞。
- 第三方版本没有变化时，不必因为 Ratis 业务代码变化而重复执行耗时的 shading。
- 集中管理版本、许可证、NOTICE、资源合并与兼容性。
- 核心模块直接依赖已经处理好的 ThirdParty 构件，边界更清晰。

#### 对你第 7 题答案的补全

“防止使用方发生 Maven 冲突”方向正确，但更精确的目标是避免运行时 classpath 的类名碰撞和二进制不兼容。独立发布还带来复用、构建效率与集中治理。

### 8.3 skipShade 的准确理解

-DskipShade=true 只是在 Maven 用户属性中定义了 skipShade。它不是 Maven 内置开关。只有 POM 或插件配置读取该属性，例如 `<skip>${skipShade}</skip>`，它才产生效果。当前 Ratis 核心 POM 不读取它，因此随手添加没有作用；历史文档中的语境不能直接套用到当前构建。

## 9. 你的薄弱点与纠偏清单

| 容易混淆的说法                        | 更准确的理解                                          |
|--------------------------------|-------------------------------------------------|
| compile scope 的依赖会被打进 JAR      | scope 决定依赖在哪些 classpath 可见；普通 JAR 是否合并依赖由打包插件决定 |
| -am 把 library 打进 app JAR       | -am 让依赖模块加入当前 Reactor，并为本次构建提供输出                |
| app 编译会重新验证 consumer-b 的所有方法调用 | app 编译器通常只验证 app 自己的源码；依赖字节码中的符号可能到运行期才解析       |
| fat JAR 解决依赖冲突                 | fat JAR 改变部署形态；版本选择仍由 Maven 解析，兼容性仍需保证          |
| relocation 后是两个 Maven 依赖       | 关键是两个不同 FQCN；relocated 类可能已经嵌入另一个构件             |
| 两个不同坐标不会冲突                     | Maven 不仲裁，但它们仍可能包含相同 FQCN，引发 JVM classpath 冲突   |
| BUILD SUCCESS 表示应用一定能运行        | 它只证明本次构建目标成功；运行 classpath 与二进制兼容仍需验证            |

## 10. Windows 排错操作手册

### 10.1 先确认工具与当前项目

```
.\mvnw.cmd -version
.\mvnw.cmd -f .\某实验\pom.xml help:effective-pom
.\mvnw.cmd -f .\某实验\pom.xml help:effective-settings
```

### 10.2 再确认依赖解析结果

```
.\mvnw.cmd -f .\pom.xml dependency:tree -Dverbose
.\mvnw.cmd -f .\pom.xml dependency:tree -Dincludes=某个groupId:artifactId
```

依次问自己：它是直接还是传递依赖？路径多深？哪个版本 omitted？最终 scope 是什么？是否被 exclusion 排除？

### 10.3 然后检查真实 JAR

```
jar tf .\target\app.jar | Select-String 'TextFormatter|DemoLibrary'
javap -classpath .\target\app.jar -c 完整类名
```

dependency:tree 告诉你 Maven 模型；jar tf 告诉你文件事实；javap 告诉你字节码实际引用。三个证据层层深入。

## 10.4 按错误类型定位

| 现象                            | 优先检查                                            |
|-------------------------------|-------------------------------------------------|
| Could not find artifact       | 坐标、版本、仓库；同 Reactor 是否缺 -am；是否需要 install         |
| NoClassDefFoundError          | 运行 classpath 是否缺 JAR；thin JAR 是否被误当独立应用         |
| NoSuchMethodError             | 运行时加载了哪个版本；dependency:tree 是否有 omitted；是否二进制不兼容 |
| duplicate/overlapping classes | 不同 JAR 是否含相同 FQCN；是否应排除、统一版本或 relocation        |
| IDE 成功、命令行失败                  | Project SDK、Runner JRE、JAVA_HOME、Maven 版本、工作目录  |

## 11. 复习卡：必须能脱口而出的 20 题

### 1. GAV 是什么？

答案：groupId、artifactId、version。

### 2. target 为什么可删除？

答案：它是可由 POM、源码和资源重新生成的构建输出。

### 3. package 为什么自动包含 compile？

答案：它们是同一生命周期中有序阶段，执行后者会经过前置阶段。

### 4. dependency 与 plugin 的区别？

答案：前者服务项目 classpath，后者执行 Maven 构建动作。

### 5. package 与 install 的区别？

答案：package 写 target；install 还写本地仓库。

### 6. 直接依赖和传递依赖？

答案：当前 POM 声明的是直接依赖，由其带入的是传递依赖。

### 7. parent 与 modules？

答案：parent 是继承；modules 是聚合。

### 8. 当前 Reactor 是什么？

答案：本次命令选中的项目集合、依赖图与构建顺序。

### 9. -pl 与 -am？

答案：-pl 选项目；-am 同时构建其 Reactor 依赖。

### 10. -am 为什么不等于 Fat JAR？

答案：它改变参与构建的项目，不改变 jar 插件的打包规则。

### 11. Maven 仲裁依据？

答案：同一冲突身份的多个版本，近者优先，同深度先声明。

## 12. omitted for conflict?

答案：该版本在依赖图出现但仲裁落败，不进入最终解析结果。

## 13. NoSuchMethodError?

答案：类存在，但运行时加载版本缺少字节码要求的方法。

## 14. 可执行 JAR 与 Fat JAR?

答案：前者有入口，后者含依赖；是两个维度。

## 15. Fat JAR 解决什么?

答案：主要解决部署与 classpath 组装便利。

## 16. Shade 与 relocation?

答案：Shade 是重打包/转换；relocation 是其中的类名重写能力。

## 17. relocation 改写什么?

答案：class 路径、内部类名和使用者字节码符号引用。

## 18. 不同坐标包含同名 Class 怎么办?

答案：Maven 默认不仲裁；JVM 通常按 classpath 顺序加载一个。

## 19. Ratis ThirdParty 的核心价值?

答案：用私有命名空间隔离第三方类，并集中治理与复用。

## 20. skipShade 是内置开关吗?

答案：不是；只有构建配置显式读取该属性才有效。

## 12. 巩固练习与参考答案

### 练习 A：预测而不是先运行

一个 app 依赖 lib；app 的 manifest 有 Main-Class，但使用普通 jar 插件。请预测：app 能否 compile？app JAR 是否含 lib class？java -jar 是否一定成功？

### 练习 B：画出当前 Reactor

parent 聚合 api、service、app；app 依赖 service，service 依赖 api。分别写出执行 -pl app package 与 -pl app -am package 时的当前 Reactor。

### 练习 C：区分两类冲突

场景一：g:a:1.0 和 g:a:2.0。场景二：x:a:1.0 和 y:b:1.0 都包含 p.C。分别说明 Maven 与 JVM 会做什么。

### 练习 D：用证据定位 NoSuchMethodError

设计一个三步检查：先用 dependency:tree 判断版本，再用 jar tf 确认类存在，最后用 javap 检查调用方符号引用。写下每一步想证明什么。

参考答案

A：能否编译取决于 lib 是否在编译 classpath；普通 app JAR 不含 lib class；java -jar 能找到入口但可能因缺 lib 而失败。B：前者只有 app，后者包含 api、service、app（以及 Maven 需要的聚合项目）。C：场景一发生 Maven 版本仲裁；场景二 Maven 默认都保留，JVM/打包工具面对重复 FQCN 时通常只会实际使用或留下一个。D：三种工具分别证明 Maven 模型、归档事实和字节码事实。

### 12.1 推荐复习节奏

| 时间    | 任务                   | 完成标准                                          |
|-------|----------------------|-----------------------------------------------|
| 今天    | 通读第 1、4、5、6、7 章      | 能口述 Reactor 到 classpath 的完整链路                 |
| +1 天  | 遮住答案做 20 张复习卡        | 至少答对 16 题                                     |
| +3 天  | 重跑 Lab03-Lab05 的关键命令 | 能先预测 dependency:tree、JAR 内容与错误                |
| +7 天  | 完成四个巩固练习             | 不看笔记写出原因，不只写现象                                |
| +14 天 | 在 Ratis 中选一个模块画依赖图   | 说明 Reactor、Artifact、classpath 与 ThirdParty 边界 |

## 结语

你已经跨过 Maven 学习中最难的一道坎：不再只记命令，而是开始追问“这个 class 从哪里来、在哪个阶段出现、谁选择了它、JVM 最终加载了谁”。以后面对 Ratis 的大型多模块构建，继续坚持同一种方法：先画模型，再预测，最后用 `dependency:tree`、`jar tf` 和 `javap` 验证。

资料范围：ratis-labs 的 Lab01-Lab05、实验记录及学习过程中的实际问题。